

# Project Report

## Wildlife Information System (WIS)

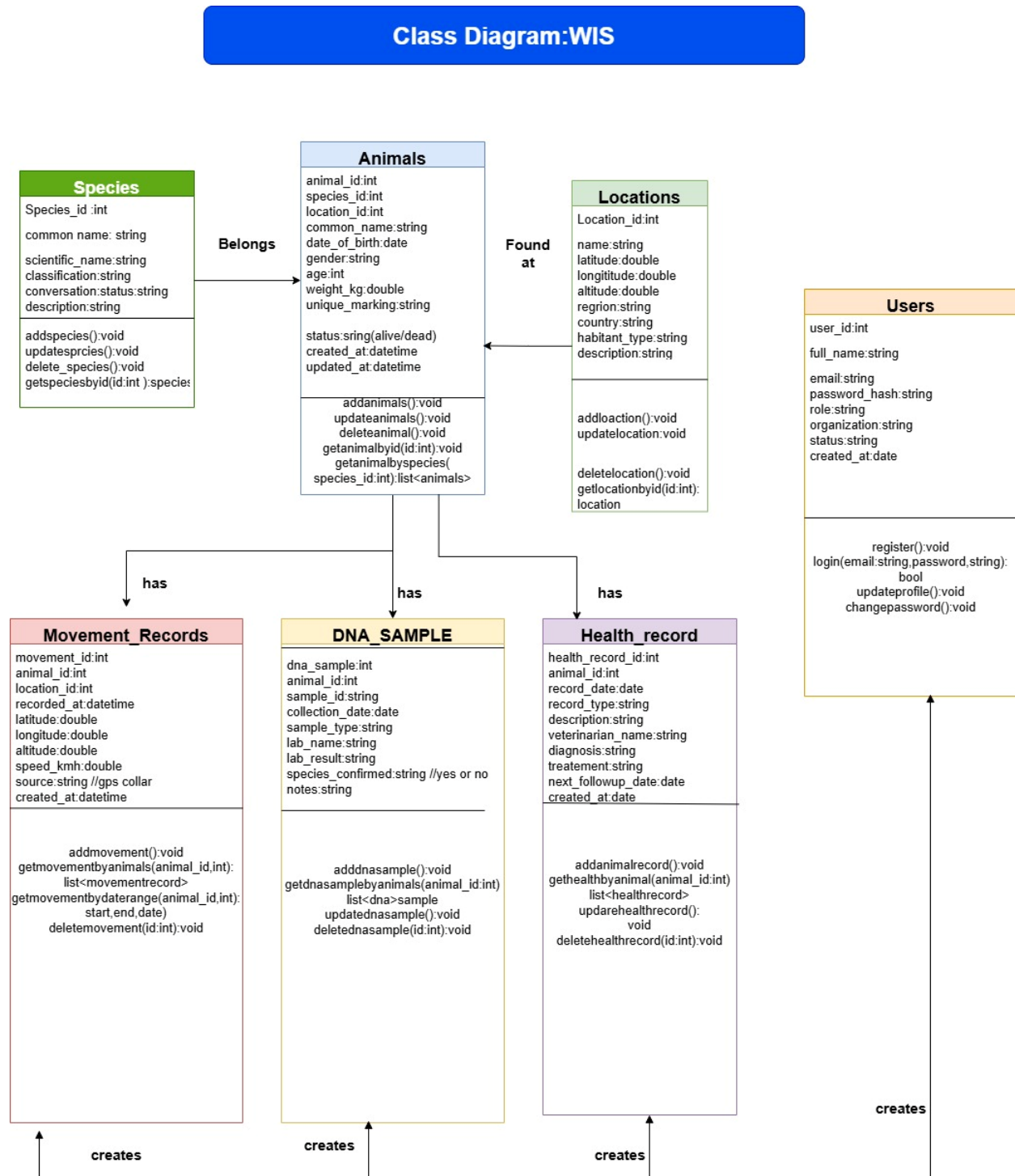
Document Number: WIS-SRS-001

**Prepared by:**  
Muhammad Ehtisham  
Fahad Naeem Khan  
Muhammad Abbas

## 4. DESIGN

### 4.1 Class Diagram

The class diagram explains the master and core classes in the system with their attributes and methods:



**Figure 9: Class Diagram**

### **Class: Animals**

**Attributes:** animalId, speciesId, name, gender, ageYears, weightKg, locationId, addedBy, addedAt

**Methods:** save(), update(), delete(), getMovementHistory(), getHealthRecords(), getDNASamples()

### **Class: Movement\_Record**

**Attributes:** recordId, animalId, latitude, longitude, recordedAt, source

**Methods:** save(), getByAnimal(), bulkInsert()

### **Class: DNA\_Sample**

**Attributes:** sampleId, animalId, collectedAt, labName, result, speciesConfirmed

**Methods:** save(), getByAnimal()

### **Class: Health\_Record**

**Attributes:** healthId, animalId, date, condition, treatment, vetName, vaccineGiven

**Methods:** save(), update(), getByAnimal()

### **Class: User**

**Attributes:** userId, name, email, passwordHash, role, createdAt

**Methods:** login(), logout(), updateProfile(), changePassword()

### **Class: Species**

**Attributes:** speciesId, commonName, scientificName, conservationStatus, habitat

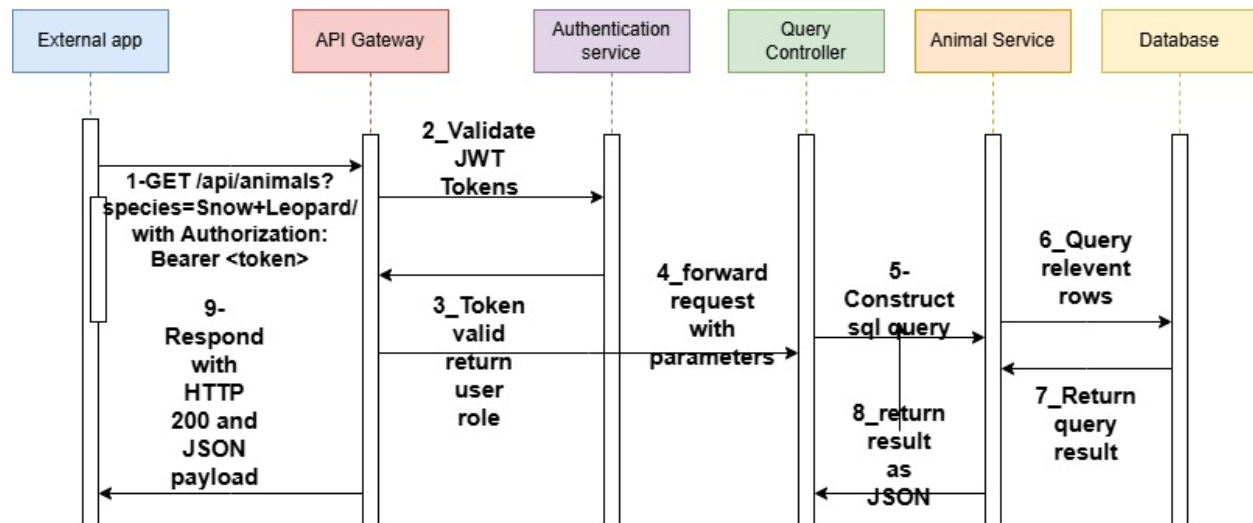
**Methods:** save(), getAll(), getByName()

## **4.2 Sequence Diagram**

The detailed sequence for "Query Animal Data via API" is explained :

1. External App sends: GET /api/animals?species=Snow+Leopard/ with Authorization: Bearer <token>
2. The API Gateway verify the JWT token.
3. The Authentication Service verify the token and provides the user role.
4. The Query Controller manages the incoming request and forwards the parameters to the Animal Service.
5. The Animal Service makes an SQL query that manages both the animals table and the species table.
6. The database identify the relevant rows.
7. The Animal Service returns back the result as JSON.
8. The API Gateway responds with an HTTP 200 and the JSON payload.

### Sequence Diagram:Query Animals Data via API



## 4.3 Architecture

The Wildlife Information System follows a 3-Tier Architecture:

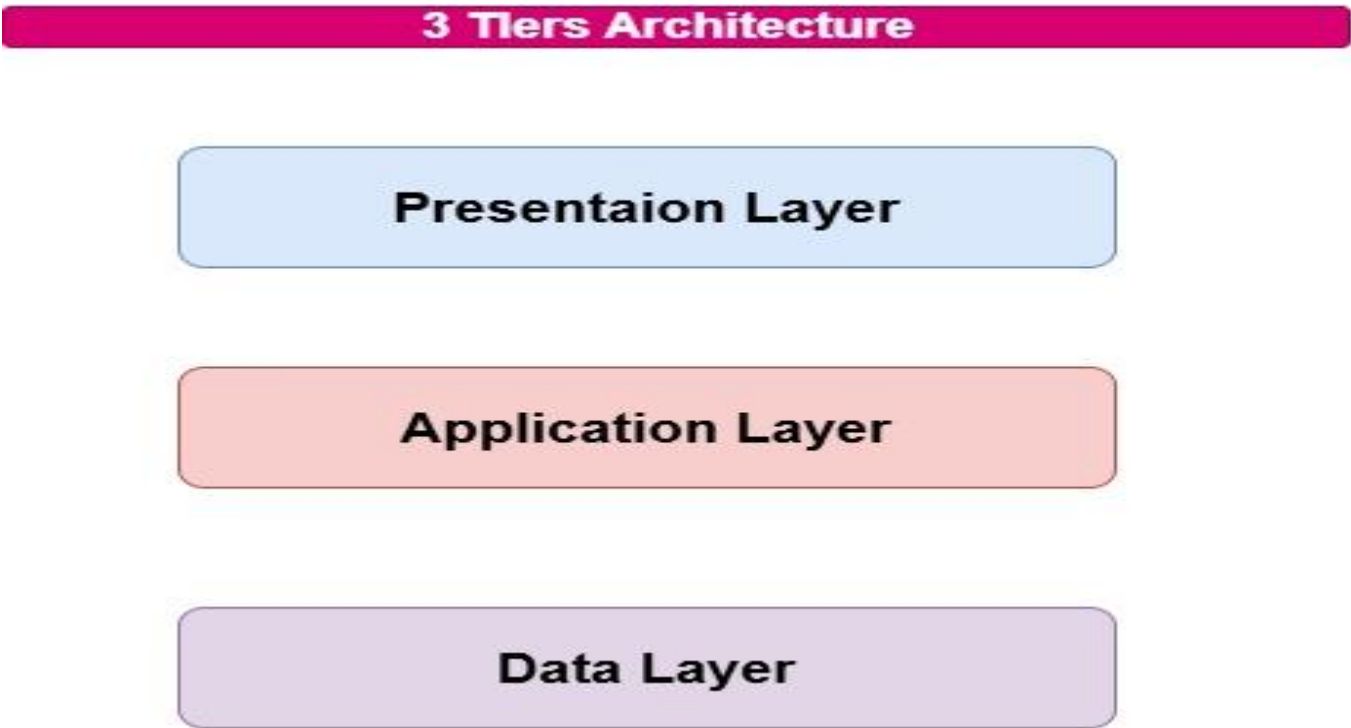


Figure 10: 3-Tier Architecture Diagram

| Layer              | Component                       | Responsibility                                     |
|--------------------|---------------------------------|----------------------------------------------------|
| Presentation Layer | HTML/CSS/JS Web Interface       | Basic UI for data entry, search, and export        |
| Application Layer  | Node.js + Express.js API Server | Business logic, authentication, query processing   |
| Data Layer         | PostgreSQL Database             | Stores all wildlife records, users, and audit logs |

External users and applications communicate with the Application Layer through the RESTful API. The Application Layer handles authentication, business logic, and data formatting. The Data Layer stores everything persistently

Key API Endpoints

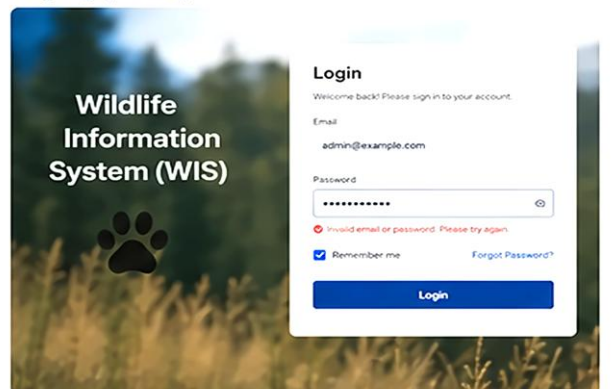
| Method | Endpoint                   | Description                            |
|--------|----------------------------|----------------------------------------|
| GET    | /api/animals               | Get list of all animals (with filters) |
| GET    | /api/animals/:id           | Get a single animal by ID              |
| POST   | /api/animals               | Add a new animal record                |
| PUT    | /api/animals/:id           | Update animal record                   |
| DELETE | /api/animals/:id           | Delete animal record                   |
| GET    | /api/animals/:id/movements | Get movement history for an animal     |
| POST   | /api/movements/upload      | Bulk upload CSV movement data          |

| Method | Endpoint                | Description                      |
|--------|-------------------------|----------------------------------|
| GET    | /api/animals/:id/health | Get health records for an animal |
| GET    | /api/animals/:id/dna    | Get DNA records for an animal    |
| POST   | /api/auth/login         | User login and get JWT token     |
| GET    | /api/species            | Get list of all species          |

## 4.4 Interface Design

This application would have an easy-to-use web-based interface. The key pages contains:

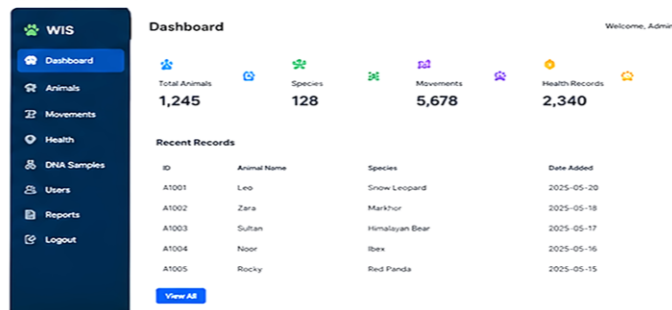
1. Login Page (Error State)



**Login Page** : Contains the email address field, password field, and login button. Displays an error message if incorrect.

**Dashboard** : Has a welcome message, statistics about animals (including total number, number of species, and some recent records), and navigation menu.

2. Dashboard



**Animal List Page** : Displays a sortable table containing all animals with View, Edit, and Delete buttons.

## 3. Animal List Page

| ID    | Name   | Species        | Gender | Age | Location   | Actions               |
|-------|--------|----------------|--------|-----|------------|-----------------------|
| A1001 | Leo    | Snow Leopard   | M      | 5   | Gilgit     | [Edit] [Add] [Delete] |
| A1002 | Zara   | Markhor        | F      | 3   | Chitral    | [Edit] [Add] [Delete] |
| A1003 | Sultan | Himalayan Bear | M      | 6   | Skardu     | [Edit] [Add] [Delete] |
| A1004 | Noor   | Ibex           | F      | 4   | Muzee      | [Edit] [Add] [Delete] |
| A1005 | Rocky  | Red Panda      | M      | 2   | Islamabad  | [Edit] [Add] [Delete] |
| A1006 | Beta   | Brown Bear     | F      | 7   | Hunza      | [Edit] [Add] [Delete] |
| A1007 | Shadow | Wolf           | M      | 4   | Astore     | [Edit] [Add] [Delete] |
| A1008 | Simba  | Lion           | M      | 6   | Bahawalpur | [Edit] [Add] [Delete] |

**Add Animal Form :** Contains fields for entering information on the species, name, gender, age, weight, and location of an animal. Includes a submission button for recording the data.

## 4. Add Animal Form

**Movement Records Page:** Displays a sortable table containing GPS coordinates and the time stamps for a particular animal with an option to add a CSV file.

## 5. Movement Records Page

| Record ID | Latitude | Longitude | Recorded At         |
|-----------|----------|-----------|---------------------|
| MS001     | 35.1234  | 74.5678   | 2025-05-20 08:10:30 |
| MS002     | 35.2234  | 74.6678   | 2025-05-20 12:45:20 |
| MS003     | 35.3234  | 74.7678   | 2025-05-20 16:10:50 |
| MS004     | 35.4234  | 74.8678   | 2025-05-20 20:30:15 |
| MS005     | 35.5234  | 74.9678   | 2025-05-21 00:50:05 |

**API Documentation Page:** A straightforward page listing APIs, required parameters, and example responses.

**User Management (Admin Only):** Displays a sortable table containing information on all the users registered with options to create or remove an account.

**Design principles:** Minimal use of color; the only colors being used are blue and white. All buttons have descriptive names. Errors are displayed in red. Success messages are displayed in green.

6. API Documentation Page

API Documentation

Base URL  
https://api.wis.com/v1

Endpoints

| Method | Endpoint      | Description                      |
|--------|---------------|----------------------------------|
| GET    | /animals      | Get list of all animals          |
| GET    | /animals/{id} | Get details of a specific animal |
| POST   | /animals      | Create a new animal              |
| PUT    | /animals/{id} | Update an existing animal        |
| DELETE | /animals/{id} | Delete an animal                 |

Example: Get Animal Details

GET /animals/{id}

Parameters

| Name | Type    | Description |
|------|---------|-------------|
| id   | integer | Animal ID   |

Response (200 OK)

```
{  "id": "A1001",  "name": "Leo",  "species": "Snow Leopard",  "gender": "M",  "age": 5,  "weight": 45,  "location": "Hilltop"}
```

7. User Management (Admin Only)

WIS

Dashboard

Animals

Movements

Health

DNA Samples

Users

Reports

Logout

Users

+ Create User

| ID   | Full Name  | Email                | Role          | Status   | Created At          | Actions                                      |
|------|------------|----------------------|---------------|----------|---------------------|----------------------------------------------|
| U001 | Admin User | admin@example.com    | Administrator | Active   | 2025-01-10 09:30:00 | <div><div></div><div></div><div></div></div> |
| U002 | Ranger One | ranger1@example.com  | Ranger        | Active   | 2025-01-12 10:15:20 | <div><div></div><div></div><div></div></div> |
| U003 | Vet Doctor | vet@example.com      | Veterinarian  | Active   | 2025-01-14 11:45:10 | <div><div></div><div></div><div></div></div> |
| U004 | Researcher | research@example.com | Researcher    | Inactive | 2025-01-18 14:20:30 | <div><div></div><div></div><div></div></div> |
| U005 | Viewer     | viewer@example.com   | Viewer        | Active   | 2025-01-20 16:05:40 | <div><div></div><div></div><div></div></div> |

Showing 1 to 5 of 25 users

1

2

3

4

5

...

Create User

Full Name

Enter full name

Email

Enter email

Role

Select Role

Password

Enter password

Confirm Password

Confirm password

Status

Active

Create User

Cancel

Edit User (U002)

Full Name

Ranger One

Email

ranger1@example.com

Role

Ranger

Status

Active

Update User

Cancel